PatientTriage.ai - Round 2 Prototype README

PatientTriage.ai — Round 2 Prototype

Accenture Innovation Challenge 2026 — Problem Track 2

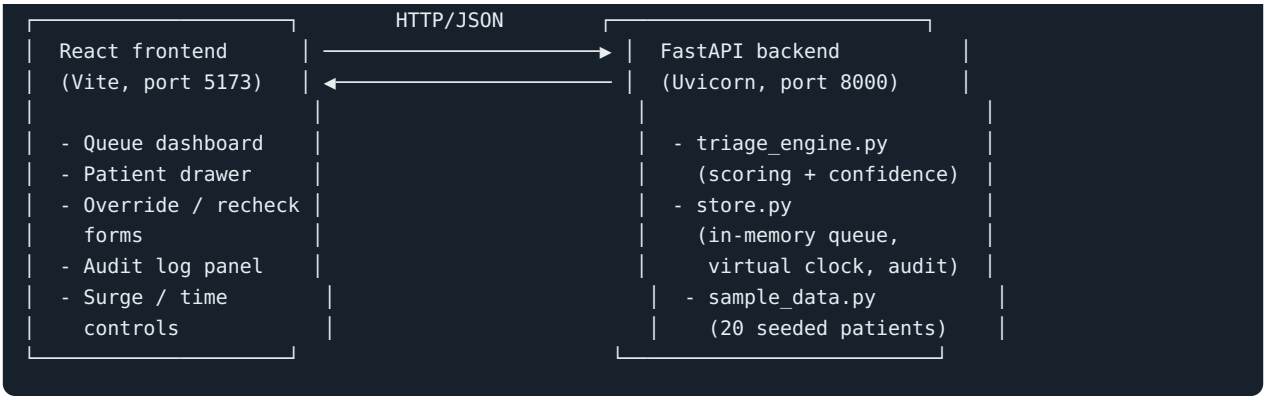
A continuous, uncertainty-aware triage assistant for hospital emergency departments. Instead of scoring a patient once at arrival and stopping, the system keeps watching the whole waiting room — rechecking patients on an interval scaled to their risk, escalating automatically when a wait exceeds a safe threshold, and falling back to simple deterministic rules rather than guessing when the system is under surge or data is too thin to trust.

Assumed regulatory jurisdiction: HIPAA (United States). This affects the audit trail design below (every score, override, and escalation is logged with who/what/when/why) and the assumption that patient records here are synthetic, not real PHI.

Why this design (recap from Round 1 + Round 2 additions)

Requirement	How the prototype addresses it
Decide vs. recommend	The engine only ever produces a score + confidence band and can flag/escalate. A nurse's override is the only thing that changes a patient's actual queue position with full clinical authority — and every override requires a typed reason.
Realistic first-minute data	Vitals fields are nullable throughout. The scoring engine explicitly handles missing vitals rather than assuming complete data (see `Ravi Menon` and `Deepak Joshi` in the seed data — both score on a chief complaint with little or no vitals).
Worst case, not average case	Two explicit worst-case paths: (1) low-confidence cases are escalated one level rather than left at face value, and (2) a surge (≥3× normal queue capacity) trips **fallback mode**, which reverts to a simple deterministic ESI rule table instead of trusting the fuller confidence-weighted model on a system that may be getting degraded or delayed data.
Age-banded thresholds	Vital sign thresholds are defined separately for pediatric (<12), adult (12–65), and geriatric (>65) patients — see `VITAL_THRESHOLDS` in `triage_engine.py`. The same raw heart rate or temperature can be flagged differently depending on age band.
Continuous monitoring of the queue	The virtual clock (`advance_time`) re-checks every waiting patient against a safe-wait threshold for their current ESI level and auto-escalates if they've been waiting too long without a recheck. Vitals rechecks (`update_vitals`) also detect and log worsening/improving trends.
Escalation bias under uncertainty	Implemented explicitly in `score_patient()` : if confidence is below 0.5 and the computed level is 3 or worse, the engine escalates one level rather than leaving a low-confidence read at low priority.
Audit trail	Every automatic score, override, and escalation is appended to an in-memory audit log with a timestamp, actor, reason, and before/after level. Visible in the UI's "Audit log" panel.

Architecture



The backend holds all state in memory (`store.py`) for this prototype — there is no database, since the brief does not require persistence and an in-memory store keeps the core mechanism easy to inspect and demo. A production version would replace `TriageStore` with a real database and add authentication; the scoring and monitoring logic in `triage_engine.py` would carry over unchanged.

Data flow for one patient

1. **Intake** — `POST /api/patients` with whatever fields are available (vitals may be null).
2. **Scoring** — `score_patient()` computes an ESI-aligned level (1–5) and a confidence band, using age-banded thresholds and a bias toward escalation under uncertainty.
3. **Confidence gate** — if confidence is "low", the patient's status is set to `in_review` (routed straight to a nurse) instead of `waiting`.
4. **Continuous watch** — while `waiting`, `advance_time()` checks elapsed wait against `SAFE_WAIT_THRESHOLDS` for the patient's level and auto-escalates if exceeded.
5. **Recheck** — `update_vitals()` re-scores with updated vitals and records whether the trend is worsening, improving, or stable.
6. **Override** — a nurse can set any ESI level at any time via `POST /api/patients/{id}/override`; this requires a reason and is logged.
7. **Surge** — `POST /api/simulate-surge` adds a burst of patients; once load exceeds 3× normal capacity, all new scoring (including on existing patients being rechecked) uses fallback mode.

Repository structure

```
.
├── backend/
│   ├── app/
│   │   ├── main.py            FastAPI app + routes
│   │   ├── triage_engine.py   Scoring logic (age bands, confidence, fallback)
│   │   ├── store.py           In-memory queue, virtual clock, audit log
│   │   └── sample_data.py     20 simulated patient records
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── App.jsx
│   │   ├── api.js             API client
│   │   └── components/        QueueTable, PatientDrawer, ControlBar, AuditLog, WalkInForm, Badges
│   └── package.json
└── README.md
```

Dependencies

Backend: Python 3.10+, FastAPI 0.141, Uvicorn 0.52, Pydantic 2.13 (see `backend/requirements.txt`)

Frontend: Node.js 18+, React 19, Vite, Axios (see `frontend/package.json`)

Running the prototype

1. Backend

```
cd backend
python3 -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --port 8000
```

The API is now live at `http://localhost:8000`. Interactive API docs (Swagger UI) are auto-generated at `http://localhost:8000/docs`.

2. Frontend

In a second terminal:

```
cd frontend
npm install
npm run dev
```

Open `http://localhost:5173` in a browser. The dashboard seeds itself with 20 simulated patients on backend startup.

Demo checklist (matches the brief’s minimum prototype expectations)

- ☒ Triage scoring on 20 simulated patient records (exceeds the 15–20 minimum)
- ☒ Ambiguous presentation — **Ravi Menon** (vague chest discomfort, one partial vital)
- ☒ Pediatric case — **Aanya Kapoor** (age 4), **Ishaan Verma** (age 2)
- ☒ Geriatric case — **Fatima Sheikh** (age 78), **Joseph Mathew** (age 82)
- ☒ Zero-history first-time patient — **Priya Nair**, **Deepak Joshi**
- ☒ Behavior under a simulated 3× surge — click **“Simulate surge (3×)”** in the UI; watch the mode indicator switch to “Fallback mode” and new/rechecked patients get scored via the deterministic rule table
- ☒ Every score carries an explicit confidence indicator — visible as the colored confidence meter on every row and in the patient drawer
- ☒ At least one clinician override, logged — use the **Override** tab in any patient’s drawer; check the **Audit log** panel to see it recorded with reason, actor, and before/after level

To see continuous monitoring in action: click **“Advance +60 min”** a couple of times and watch lower-priority patients who’ve waited too long automatically escalate, each logged in the audit trail.

Known limitations (honest, by design)

This is a Round 2 proof-of-concept, not a production system. Notably: - State is in-memory and resets when the backend restarts — no persistence layer yet. - No authentication — a real deployment would need role-based access control before any nurse-facing override capability went live. - Vital-sign thresholds are illustrative reference ranges for demonstration, not sourced from a specific clinical guideline document — a real deployment would need clinical sign-off on exact thresholds. - The “fallback mode” rule table is intentionally simple; a production fallback would likely be the hospital’s existing paper/EHR-based ESI process, not a second software model.